

Μη σχεσιακές βάσεις δεδομένων

Από τους πίνακες στα έγγραφα

Εβδομάδα 2

Μέρος 3ο



Αθανάσιος Χριστόπουλος, Ph.D.

Αν. Καθηγητής, Τμήμα Φυσικών Επιστημών. Κέντρο Μαθησιακής Αναλυτικής, Πανεπιστήμιο του Τούρκου, Φινλανδία
atchri@utu.fi

Τι περιέχει το Μέρος 3;

- 1 Ενημέρωση εγγράφων
updateOne, updateMany, upsert
- 2 Διαγραφή πολλών εγγράφων, με μέτρηση πριν από τη διαγραφή.
- 3 Συνάθροιση
ροή σταδίων, στις συλλογές country, books και persons
- 4 Οι εντολές της SQL στη MongoDB, και η Εργασία 2 ενότητα προς ενότητα.

Οι ενημερώσεις γίνονται στη συλλογή country: Ελλάδα, Γαλλία, Γερμανία. Όποιος δεν την έχει, επικολλά στο mongosh το country.txt της πλατφόρμας.

Οι τελεστές ενημέρωσης

Τελεστής	Τι κάνει;
<code>\$set</code>	ορίζει την τιμή ενός πεδίου, και το δημιουργεί αν λείπει
<code>\$unset</code>	αφαιρεί ένα πεδίο
<code>\$inc</code>	αυξάνει έναν αριθμό κατά μια τιμή
<code>\$rename</code>	μετονομάζει ένα πεδίο
<code>\$push</code>	προσθέτει στοιχείο σε λίστα
<code>\$addToSet</code>	προσθέτει στοιχείο σε λίστα, μόνο αν δεν υπάρχει ήδη
<code>\$pull</code>	αφαιρεί από λίστα όσα στοιχεία ταιριάζουν σε μια συνθήκη

updateOne και \$set

Εντολές

```
use world
db.country.updateOne(
  { name: "Greece" },
  { $set: { capital: { name: "Athens", timezone: "Europe/Athens" } } }
)
```

Αποτελέσματα

```
switched to db world
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

- **Πρώτο όρισμα:** το φίλτρο. **Δεύτερο:** η αλλαγή, με τελεστή.
- Το `$set` **προσθέτει** το πεδίο capital. Αν υπήρχε, **θα άλλαζε την τιμή** του.
- Εργασία 2, ενότητα 11.

matchedCount και modifiedCount

Η ίδια εντολή, δεύτερη φορά

```
db.country.updateOne(
  { name: "Greece" },
  { $set: { capital: { name: "Athens", timezone: "Europe/Athens" } } }
)
```

Αποτέλεσμα

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
```

- **Βρέθηκε ένα έγγραφο** (matchedCount: 1), αλλά **δεν άλλαξε** (modifiedCount: 0): η τιμή ήταν ήδη αυτή.
- Με matchedCount: 0, το φίλτρο **δεν βρήκε τίποτα**: π.χ. "greece" με πεζό.

\$inc και \$push

Εντολές

```
db.country.updateOne(
  { name: "Greece" },
  { $inc: { visits: 1 }, $push: { languages: "Greek" } }
)
db.country.findOne({ name: "Greece" }, { _id: 0 })
```

Αποτελέσματα

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
{
  name: 'Greece',
  euSince: 1981,
  population: 10400000,
  capital: { name: 'Athens', timezone: 'Europe/Athens' },
  languages: [ 'Greek' ],
  visits: 1
}
```

- Η Ελλάδα δεν είχε visits: το `$inc` **το δημιουργεί** με τιμή 1.
- Ούτε λίστα languages είχε: το `$push` **τη δημιουργεί** με ένα στοιχείο.
- **Δύο τελεστές** στην **ίδια ενημέρωση**, ένα `modifiedCount: 1`.

\$unset: αφαίρεση πεδίου

Εντολή

```
db.country.updateOne({ name: "Greece" }, { $unset: { visits: "" } })
```

Αποτέλεσμα

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

- Το πεδίο visits **αφαιρείται**. Η τιμή μετά το όνομα **δεν έχει σημασία**, και συνήθως γράφεται `""`.
- Με `updateMany`, το ίδιο πεδίο αφαιρείται από **όλα τα έγγραφα** που ταιριάζουν.



Προσοχή

updateOne: μόνο το πρώτο έγγραφο

Εντολές

```
db.country.updateOne({ euSince: 1958 }, { $set: { founder: true } })
db.country.countDocuments({ founder: true })
db.country.updateMany({ euSince: 1958 }, { $set: { founder: true } })
```

Αποτελέσματα

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
1
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 1,
  upsertedCount: 0
}
```

- Η Γαλλία και η Γερμανία έχουν euSince 1958, αλλά το updateOne άλλαξε **μόνο την πρώτη**.
- Το updateMany βρήκε **και τις δύο** (matchedCount: 2) και άλλαξε όποια δεν είχε ήδη την τιμή (modifiedCount: 1).

updateMany: από «0 σελίδες» σε «άγνωστο»

Εντολές

```
use library
db.books.updateMany({ pageCount: 0 }, { $unset: { pageCount: "" } })
db.books.countDocuments({ pageCount: { $exists: false } })
```

Αποτελέσματα

```
switched to db library
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 166,
  modifiedCount: 166,
  upsertedCount: 0
}
166
```

- **Ένα φίλτρο**, και τα **166 βιβλία** με 0 σελίδες. Το `$exists: false` μετρά όσα δεν έχουν πια `pageCount`.
- Τώρα το «άγνωστο» είναι **πεδίο που λείπει**, όχι **ψεύτικο 0**: δεν μετρά στους μέσους όρους.
- **Επαναφορά της συλλογής**: ξανά `mongoimport ... --drop`, μετά τη συνάθροιση.

upsert: ενημέρωση ή δημιουργία

Εντολές

```
use world
db.country.updateOne(
  { name: "Cyprus" },
  { $set: { population: 980000 } },
  { upsert: true }
)
db.country.findOne({ name: "Cyprus" }, { _id: 0 })
```

Αποτελέσματα

```
switched to db world
{
  acknowledged: true,
  insertedId: ObjectId('6ac04e681c7ab1eedbf42746'),
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 1
}
{ name: 'Cyprus', population: 980000 }
```

- Κανένα έγγραφο δεν ταίριαξε: με `upsert: true` **δημιουργείται νέο**.
- Το νέο έγγραφο παίρνει το name **από το φίλτρο** και το population από το `$set`.

● Ερώτημα

Τι κάνει αυτή η εντολή;

Η Ελλάδα έχει ήδη και άλλα πεδία εκτός από το population.

```
db.country.updateOne({ name: "Greece" },  
                      { population: 10500000 })
```

- A** Αλλάζει μόνο τον πληθυσμό
- B** Αντικαθιστά όλο το έγγραφο
- Γ** Εμφανίζει σφάλμα
- Δ** Δεν κάνει τίποτε, χωρίς μήνυμα

01:00

Η απάντηση: σφάλμα

Εντολή

```
db.country.updateOne({ name: "Greece" },  
                     { population: 10500000 })
```

Αποτέλεσμα

```
MongoInvalidArgumentError: Update document requires atomic operators
```

- Το `updateOne` **θέλει τελεστή**: `$set`, `$inc`, `$unset` ...
- Για **αντικατάσταση ολόκληρου του εγγράφου** υπάρχει το `replaceOne`.
- **Προσοχή στο παλαιότερο υλικό**: το `update()` χωρίς τελεστή αντικαθιστούσε όλο το έγγραφο.

Μέτρηση πριν από τη διαγραφή

Εντολές

```
db.country.countDocuments({ population: { $lt: 1000000 } })  
db.country.deleteMany({ population: { $lt: 1000000 } })
```

Αποτελέσματα

```
1  
{ acknowledged: true, deletedCount: 1 }
```

- Πρώτα `countDocuments` με **το φίλτρο της διαγραφής**: 1, η Κύπρος.
- Μετά `deleteMany` με **το ίδιο φίλτρο**: `deletedCount: 1`.
- **Εργασία 2, ενότητα 12**: διαγραφή ενός εγγράφου, με `deleteOne` και φίλτρο.

Συνάθροιση: ροή σταδίων

\$match

Φιλτράρει έγγραφα, όπως το φίλτρο του find.



\$group

Ομαδοποιεί και υπολογίζει ανά ομάδα
πλήθος, άθροισμα, μέσο όρο, ελάχιστο, μέγιστο



\$sort, \$limit, \$project

Ταξινομούν, περιορίζουν και διαμορφώνουν την έξοδο.

Σύνταξη της ροής σταδίων (pipeline)

```
db.συλλογή.aggregate([  
  { στάδιο 1 },  
  { στάδιο 2 },  
  ...  
])
```

Η **έξοδος** κάθε σταδίου είναι η **είσοδος** του επόμενου.

\$group: ο συνολικός πληθυσμός

Εντολή

```
db.country.aggregate([
  { $group: { _id: null, totalPopulation: { $sum: "$population" } } }
])
```

Αποτέλεσμα

```
[ { _id: null, totalPopulation: 162600000 } ]
```

- `_id: null`: **μία μόνο ομάδα**, όλα τα έγγραφα μαζί.
- `"$population"`: με το \$ μπροστά από το όνομα, **η τιμή του πεδίου** population.
- Ελλάδα, Γαλλία και Γερμανία: **162.600.000**.

\$group ανά πεδίο: βιβλία ανά κατάσταση

Εντολές

```
use library
db.books.aggregate([
  { $group: { _id: "$status", books: { $sum: 1 } } }
])
```

Αποτελέσματα

```
switched to db library
[ { _id: 'PUBLISH', books: 363 }, { _id: 'MEAP', books: 68 } ]
```

- `_id: "$status"`: μία ομάδα **για κάθε τιμή** του status.
- `$sum: 1`: **μετρά τα έγγραφα** κάθε ομάδας, όπως το COUNT(*) με GROUP BY.
- Εργασία 2, ενότητα 10.

\$match και υπολογισμοί ανά ομάδα

Εντολή

```
db.books.aggregate([
  { $match: { categories: "Java" } },
  { $group: { _id: "$status", books: { $sum: 1 },
    avgPages: { $avg: "$pageCount" },
    maxPages: { $max: "$pageCount" } } }
])
```

Αποτέλεσμα

```
[
  { _id: 'MEAP', books: 1, avgPages: 700, maxPages: 700 },
  {
    _id: 'PUBLISH',
    books: 95,
    avgPages: 511.4868421052632,
    maxPages: 1088
  }
]
```

- `$match` **πρώτο**: μόνο τα βιβλία Java, όπως το WHERE.
- `$avg` και `$max`: **μέσος όρος και μέγιστο** ανά ομάδα.
- Το `$avg` **αγνοεί** τα βιβλία που δεν έχουν πλέον pageCount.

\$unwind: ένα έγγραφο ανά κατηγορία

Εντολή

```
db.books.aggregate([
  { $unwind: "$categories" },
  { $group: { _id: "$categories", books: { $sum: 1 } } },
  { $sort: { books: -1 } },
  { $limit: 5 }
])
```

Αποτέλεσμα

```
[
  { _id: 'Java', books: 96 },
  { _id: 'Internet', books: 41 },
  { _id: 'Microsoft .NET', books: 34 },
  { _id: 'Web Development', books: 17 },
  { _id: 'Software Engineering', books: 16 }
]
```

- `$unwind`: ένα έγγραφο για **κάθε κατηγορία κάθε βιβλίου**.
- Μετά **ομάδα ανά κατηγορία, ταξινόμηση, τα πέντε πρώτα**
- Βιβλία **χωρίς κατηγορία δεν περνούν** από το `$unwind`.

\$match μετά το \$group: το HAVING

Εντολή

```
db.books.aggregate([
  { $unwind: "$categories" },
  { $group: { _id: "$categories", books: { $sum: 1 } } },
  { $match: { books: { $gte: 12 } } },
  { $sort: { books: -1, _id: 1 } }
])
```

Αποτέλεσμα

```
[
  { _id: 'Java', books: 96 },
  { _id: 'Internet', books: 41 },
  { _id: 'Microsoft .NET', books: 34 },
  { _id: 'Web Development', books: 17 },
  { _id: 'Software Engineering', books: 16 },
  { _id: 'Business', books: 12 },
  { _id: 'Programming', books: 12 }
]
```

- `$match` μετά το `$group` **φιλτράρει ομάδες**, όπως το HAVING της SQL.
- Κατηγορίες με τουλάχιστον 12 βιβλία: **επτά**.
- `_id: 1` στο `$sort`: **σταθερή σειρά** στις ισοπαλίες.

\$project: το σχήμα της εξόδου

Εντολή

```
db.books.aggregate([
  { $match: { categories: "Java" } },
  { $group: { _id: "$status", books: { $sum: 1 },
    avgPages: { $avg: "$pageCount" } } },
  { $project: { _id: 0, status: "$_id", books: 1,
    avgPages: { $round: ["$avgPages", 0] } } }
])
```

Αποτέλεσμα

```
[
  { books: 1, status: 'MEAP', avgPages: 700 },
  { books: 95, status: 'PUBLISH', avgPages: 511 }
]
```

- `$project`: **ποια πεδία** εμφανίζονται και με **ποιο όνομα**. Εδώ το `_id` γίνεται `status`.
- `$round`: **στρογγυλοποίηση** του μέσου όρου.
- Το `$project` αντιστοιχεί στο `SELECT` της SQL.

To persons.txt στο mongosh

Το αρχείο persons.txt

```
db = db.getSiblingDB("book-filtered-top-subset");
db.persons.drop();
db.persons.createIndex({"vocation": 1, "dateofbirth": 1});
db.persons.insertMany([ ... έξι άτομα ... ]);
```

Αποτέλεσμα της τελευταίας εντολής

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ac04e6959cee8f5b3f61b8c'),
    '1': ObjectId('6ac04e6959cee8f5b3f61b8d'),
    '2': ObjectId('6ac04e6959cee8f5b3f61b8e'),
    '3': ObjectId('6ac04e6959cee8f5b3f61b8f'),
    '4': ObjectId('6ac04e6959cee8f5b3f61b90'),
    '5': ObjectId('6ac04e6959cee8f5b3f61b91')
  }
}
```

- Ανοίξτε το persons.txt, από την πλατφόρμα του μαθήματος, αντιγράψτε **όλο το περιεχόμενο** και επικολλήστε το στο mongosh. Αν το τερματικό ζητήσει επιβεβαίωση για πολλές γραμμές, επιβεβαιώστε.
- Νέα βάση με έξι άτομα. Η διεύθυνση κάθε ατόμου είναι **εμφωλευμένο έγγραφο**.
- **Εναλλακτικά, στο τερματικό:**
cd στον φάκελο του αρχείου, mongosh, και load("persons.txt"). Το load τυπώνει μόνο true.

Οι τρεις νεότεροι μηχανικοί

Εντολές

```
var pipeline = [  
  { $match: { vocation: "ENGINEER" } },  
  { $sort: { dateofbirth: -1 } },  
  { $limit: 3 },  
  { $unset: [ "_id", "vocation", "address" ] }  
]  
db.persons.aggregate(pipeline)
```

Αποτέλεσμα

```
[  
  {  
    person_id: '7363626383',  
    firstname: 'Carl',  
    lastname: 'Simmons',  
    dateofbirth: ISODate('1998-12-26T13:13:55.000Z')  
  },  
  {  
    person_id: '1723338115',  
    firstname: 'Olive',  
    lastname: 'Ranieri',  
    dateofbirth: ISODate('1985-05-12T23:14:30.000Z'),  
    gender: 'FEMALE'  
  },  
  {  
    person_id: '6392529400',  
    firstname: 'Elise',  
    lastname: 'Smith',  
    dateofbirth: ISODate('1972-01-13T09:32:07.000Z')  
  }  
]
```

Φιλτράρει τους μηχανικούς, ταξινομεί από τον νεότερο, κρατά τρεις και αφαιρεί τρία πεδία.

\$unset ως στάδιο

```
// με $unset
{ $unset: [ "_id", "vocation", "address" ] }

// με $project
{ $project: {
  _id: 0, vocation: 0, address: 0 } }
```

- Και τα δύο δίνουν **τους ίδιους** τρεις μηχανικούς.
- `$unset`: γράφει **μόνο όσα αφαιρούνται**.
- `$project`: και για **προσθήκη ή μετονομασία** πεδίων, όπως το status.

Η ενότητα 10 ζητά ομαδοποίηση

Ερώτημα με pipeline που παρουσιάζει συγκεντρωτικά, **ομαδοποιημένα αποτελέσματα**.

Εργασία 2, ενότητα 10

Το παράδειγμα της συλλογής persons

Με \$match, \$sort, \$limit και \$unset, το pipeline φιλτράρει και ταξινομεί, αλλά **δεν ομαδοποιεί**. Το ίδιο δίνει και ένα find με sort και limit.

Για την Εργασία 2

Τουλάχιστον ένα \$group, με τα **δικά σας δεδομένα**.

Ομαδοποίηση στη συλλογή persons: άτομα ανά επάγγελμα

Εντολή

```
db.persons.aggregate([
  { $group: { _id: "$vocation", people: { $sum: 1 },
    oldest: { $min: "$dateofbirth" } } },
  { $sort: { people: -1, _id: 1 } }
])
```

Αποτέλεσμα

```
[
  {
    _id: 'ENGINEER',
    people: 4,
    oldest: ISODate('1959-07-06T17:35:45.000Z')
  },
  {
    _id: 'FLORIST',
    people: 1,
    oldest: ISODate('1941-04-07T22:11:52.000Z')
  },
  {
    _id: 'POLITICIAN',
    people: 1,
    oldest: ISODate('1991-11-23T16:53:56.000Z')
  }
]
```

Μία ομάδα ανά επάγγελμα: πόσα άτομα, και η παλαιότερη ημερομηνία γέννησης (`$min`).

SQL και MongoDB: εντολές

SQL	MongoDB
INSERT INTO ... VALUES ...	insertOne, insertMany
SELECT ... FROM ... WHERE ...	find(φίλτρο, προβολή)
UPDATE ... SET ... WHERE ...	updateOne, updateMany με \$set
SET age = age + 3	\$inc: { age: 3 }
DELETE FROM ... WHERE ...	deleteOne, deleteMany με φίλτρο
DELETE FROM people	deleteMany({})
ALTER TABLE ... ADD ...	updateMany({}, { \$set: ... })
ALTER TABLE ... DROP COLUMN ...	updateMany({}, { \$unset: ... })

Στη MongoDB δεν υπάρχει ALTER TABLE: εξ ορισμού η συλλογή **δεν επιβάλλει δομή**, και τα πεδία αλλάζουν στο επίπεδο του εγγράφου.

Εργασία 2: εντολές ανά ενότητα

Ενότητα της εκφώνησης	Εντολές
1–3. Βάση, συλλογή, έγγραφα	<code>use</code> , <code>createCollection</code> , <code>insertOne</code> , <code>insertMany</code> , εμφωλευμένο: <code>capital: { ... }</code>
4. Προβολή πεδίων	<code>find(φίλτρο, { title: 1, _id: 0 })</code>
5. Λογικοί τελεστές	<code>\$and</code> , <code>\$or</code> , <code>\$nor</code> , <code>\$not</code> , ή κόμμα για το AND
6. Τελεστές σύγκρισης	<code>\$eq</code> , <code>\$ne</code> , <code>\$gt</code> , <code>\$gte</code> , <code>\$lt</code> , <code>\$lte</code> , <code>\$in</code> , <code>\$nin</code>
7. Regex	<code>{ title: /^Java/ }</code> , με <code>^</code> , <code>\$</code> , <code>.</code> και <code>i</code>
8. Σύνθετα ερωτήματα	<code>\$or</code> ή <code>\$and</code> μαζί με τελεστές σύγκρισης
9. Ταξινόμηση και περιορισμός	<code>.sort({ πεδίο: 1 })</code> , <code>.sort({ πεδίο: -1 })</code> , <code>.limit(n)</code>
10. Pipeline με ομαδοποίηση	<code>aggregate</code> με <code>\$match</code> , <code>\$group</code> , <code>\$sort</code> , <code>\$project</code>
11. Ενημέρωση	<code>updateOne(φίλτρο, { \$set: { πεδίο: τιμή } })</code>
12. Διαγραφή	<code>deleteOne(φίλτρο)</code> , με <code>countDocuments</code> πριν
JSON και μεταβλητές	<code>.forEach(printjson)</code> , <code>var φίλτρο = { ... }</code>

Εργασία 2: τι παραδίδετε;

Αναφορά

Ονοματεπώνυμο, email, τίτλος. Περιγραφή των τροποποιήσεων της βάσης της Εργασίας 1 και των πινάκων (**τουλάχιστον 2**, ο ένας **εμφωλευμένος** στον άλλον).

Ενότητες 1 έως 12

Αριθμημένες όπως στην εκφώνηση. Για κάθε κώδικα, **μία παράγραφος** που εξηγεί τι κάνει και **στιγμιότυπο οθόνης** με το αποτέλεσμα.

JSON και μεταβλητές

Σε κάποια ερωτήματα, ευανάγνωστο JSON και μεταβλητές, με **κείμενο που το επισημαίνει**.

Προθεσμία

Στην πλατφόρμα του μαθήματος.

Βασικά συμπεράσματα

- 1 Κάθε ενημέρωση έχει **φίλτρο και τελεστή**: το `updateOne` αλλάζει το πρώτο έγγραφο, το `updateMany` όλα, και χωρίς τελεστή εμφανίζεται σφάλμα.
- 2 Η συνάθροιση είναι **ροή σταδίων**: το `$match` φιλτράρει, το `$group` ομαδοποιεί, και τα `$sort`, `$limit` και `$project` διαμορφώνουν την έξοδο.
- 3 Πριν από κάθε μαζική αλλαγή ή διαγραφή, **`countDocuments` με το ίδιο φίλτρο**.

Ερωτήσεις

Εδώ ολοκληρώνονται οι μη σχεσιακές βάσεις δεδομένων. Την επόμενη εβδομάδα ξεκινά η Μαθησιακή Αναλυτική.